МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«БЕЛГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ им. В. Г. Шухова»
(БГТУ им. В. Г. Шухова)

ИНСТИТУТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ
И УПРАВЛЯЮЩИХ СИСТЕМ

Кафедра программного обеспечения вычислительной
техники и автоматизированных систем

Лабораторная работа №3

по дисциплине: «Тестирование ПС»

по теме: «Автоматизация UI-тестов и построение CI/CD-пайплайна
с блокирующими quality gates»

Выполнил: студент группы ПВ-233

Алексеенко Андрей Евгеньевич

Проверил: преподаватель

Доденко Олег Павлович

Белгород, 2026 г.

Цель работы: освоить UI-автоматизацию, паттерны проектирования тестов, встраивание тестов в пайплайн непрерывной интеграции и настройку блокирующих quality gates, при которых падение тестов останавливает сборку и релиз.

Содержание

Часть 1 3
Часть 2 6
Часть 3 7
Контрольные вопросы 12
Вывод 13

Задания

Часть 1. UI-автоматизация

Дано веб-приложение – интернет-магазин (https://github.com/Sajid788/edgistify-Assignment) Можно взять код за основу (с нуля проект не нужно создавать, можно найти готовый проект в интернете. Используете код либо для frontend или для backend разработчика).

Требуется написать E2E-тесты на Playwright или Selenium для сценариев:

- регистрация пользователя;
- вход в систему;
- поиск товара;
- добавление товара в корзину;
- оформление заказа;
- выход из системы.

Часть 2. Пирамида тестирования

1. Построить пирамиду тестирования для данного приложения.
2. Обосновать распределение: сколько unit-, integration- и E2E-тестов и почему.

Часть 3.

1. Создать CI-конфигурацию.
2. Настроить триггеры тестирования.
3. Настроить quality gates.
4. Публиковать отчёты и артефакты.

Часть 1. UI-автоматизация

Дано веб-приложение – интернет-магазин (https://github.com/Sajid788/edgistify-Assignment) Можно взять код за основу (с нуля проект не нужно создавать, можно найти готовый проект в интернете. Используете код либо для frontend или для backend разработчика).

Требуется написать E2E-тесты на Playwright или Selenium для сценариев:

- регистрация пользователя;
- вход в систему;
- поиск товара;
- добавление товара в корзину;
- оформление заказа;
- выход из системы.

Для тестирования пользовательского интерфейса фронтенд-части выбран фреймворк Playwright. Тесты спроектированы с использованием паттерна Page Object Model, который позволяет отделить логику взаимодействия со страницами от самих сценариев тестов.

Управление входом и регистрацией

```
import { Page, Locator, expect } from '@playwright/test';

export class AuthPage {
  readonly page: Page;
  // Локаторы для регистрации
  readonly fullNameInput: Locator;
  readonly emailInput: Locator;
  readonly passwordInput: Locator;
  readonly signupButton: Locator;
  // Локаторы для логина
  readonly loginEmailInput: Locator;
  readonly loginPasswordInput: Locator;
  readonly loginButton: Locator;
  readonly logoutButton: Locator;

  constructor(page: Page) {
    this.page = page;
    this.fullNameInput = page.getByPlaceholder('Full Name');
    this.emailInput = page.getByPlaceholder('Email');
    this.passwordInput = page.getByPlaceholder('Password');
    this.signupButton = page.getByRole('button', { name: 'Sign Up' });

    this.loginEmailInput = page.getByPlaceholder('Email');
    this.loginPasswordInput = page.getByPlaceholder('Password');
    this.loginButton = page.getByRole('button', { name: 'Login' });
    this.logoutButton = page.getByRole('button', { name: 'Logout' });
```

```
  }

  async register(name: string, email: string, pass: string) {
    await this.page.goto('/signup');
    await this.fullNameInput.fill(name);
    await this.emailInput.fill(email);
    await this.passwordInput.fill(pass);
    await this.signupButton.click();
  }

  async login(email: string, pass: string) {
    await this.page.goto('/login');
    await this.loginEmailInput.fill(email);
    await this.loginPasswordInput.fill(pass);
    await this.loginButton.click();
    // Проверяем, что редиректнуло на главную
    await expect(this.page).not.toHaveURL(/.*login/);
  }

  async logout() {
    await this.logoutButton.click();
    await expect(this.page).toHaveURL(/.*login/);
  }
}
```

Поиск и корзина

```
import { Page, Locator, expect } from '@playwright/test';

export class ProductPage {
  readonly page: Page;
  readonly searchInput: Locator;
  readonly addToCartButton: Locator;

  constructor(page: Page) {
    this.page = page;
    this.searchInput = page.getByPlaceholder(/Search/i);
    this.addToCartButton = page.getByRole('button', { name: /Add to Cart/i });
  }

  async searchProduct(productName: string) {
    await this.searchInput.fill(productName);
    await this.searchInput.press('Enter');
    await expect(this.page.getByText(productName).first()).toBeVisible();
  }

  async addToCart(productName: string) {
    // Кликаем по карточке товара и добавляем в корзину
    await this.page.getByText(productName).first().click();
    await this.addToCartButton.click();
  }
}
```

Оформление заказа

```
import { Page, Locator, expect } from '@playwright/test';

export class CheckoutPage {
  readonly page: Page;
  readonly addressInput: Locator;
  readonly placeOrderButton: Locator;

  constructor(page: Page) {
    this.page = page;
    this.addressInput = page.getByPlaceholder(/Address/i);
    this.placeOrderButton = page.getByRole('button', { name: /Place Order/i });
  }

  async placeOrder(address: string) {
    await this.page.goto('/checkout');
    await this.addressInput.fill(address);
    await this.placeOrderButton.click();
    await expect(this.page.getByText(/Order Placed Successfully/i)).toBeVisible();
  }
}
```

Основной файл тестов

```
import { test, expect } from '@playwright/test';
import { AuthPage } from '../pages/AuthPage';
import { ProductPage } from '../pages/ProductPage';
import { CheckoutPage } from '../pages/CheckoutPage';

// Генерируем уникальный email для каждого запуска
const uniqueEmail = `user_${Date.now()}@example.com`;
const userPass = 'SecurePassword123!';

test.describe('E2E Сценарии интернет-магазина', () => {

  test('Регистрация, вход и выход из системы', async ({ page }) => {
    const auth = new AuthPage(page);

    // 1. Регистрация
    await auth.register('Test User', uniqueEmail, userPass);

    // 2. Вход в систему
    await auth.login(uniqueEmail, userPass);

    // 3. Выход из системы
    await auth.logout();
  });

  test('Поиск товара, добавление в корзину и оформление заказа', async ({ page }) => {
    const auth = new AuthPage(page);
    const product = new ProductPage(page);
    const checkout = new CheckoutPage(page);
```

```
    // Авторизация
    await auth.login(uniqueEmail, userPass);

    // 1. Поиск товара
    await product.searchProduct('Laptop');

    // 2. Добавление в корзину
    await product.addToCart('Laptop');

    // 3. Оформление заказа
    await checkout.placeOrder('123 Main St, New York, NY 10001');
  });
});
```

Часть 2. Пирамида тестирования

Задание 1. Построить пирамиду тестирования для данного приложения.

Для данного приложения пирамида тестирования выглядит следующим образом:

1) Основание (Unit-тесты) — ~70%
    - Проверка изолированных функций и компонентов.
    - Примеры для этого проекта: валидация email и пароля при регистрации, функции калькуляции стоимости корзины, проверка рендеринга React-компонента кнопки «Добавить в корзину» с нужными пропсами, логика хеширования пароля (bcrypt).
2) Середина (Integration-тесты) — ~20%
    - Проверка взаимодействия между модулями или слоями приложения.
    - Примеры для этого проекта: проверка API-эндпоинтов (например, POST /api/users/register записывает пользователя в тестовую БД), взаимодействие контроллера корзины с базой данных MongoDB, проверка того, что после оформления заказа корзина очищается.
3) Вершина (E2E-тесты) — ~10%
    - Проверка сквозных пользовательских сценариев через реальный браузер (Playwright).
    - Примеры для этого проекта: тесты, написанные в Части 1 (регистрация → вход → поиск → корзина → оформление заказа → выход).

Задание 2. Обосновать распределение: сколько unit-, integration- и E2E-тестов и почему.

Такое распределение (70/20/10) выбрано по следующим причинам:

1) Скорость и стоимость: Unit-тесты работают за миллисекунды, их можно запускать тысячи раз в день локально при каждом сохранении файла. E2E-тесты запускают реальный браузер, они медленные и требуют поднятого фронтенда и бэкенда. Если делать 70% тестов через E2E, пайплайн CI будет идти часами.

2) Хрупкость (flakiness): E2E-тесты зависят от верстки, скорости интернета и окружения. Они часто падают «на пустом месте». Unit-тесты стабильны и падают только при реальном изменении логики.

3) Точность локализации ошибки: Если упал Unit-тест калькулятора корзины, разработчик сразу понимает, в каком файле искать баг. Если упал E2E-тест на оформление заказа, ошибка может быть в верстке кнопки, в API, в базе данных или в логике связывания — искать приходится долго.

4) Специфика проекта: В интернет-магазине много сложной бизнес-логики (скидки, остатки на складе, расчет доставки). Эту логику дешевле и надежнее покрывать Unit-тестами на бэкенде (Node.js) и Unit-тестами компонентов на фронтенде (Jest/React Testing Library). E2E оставляем только для критических «счастливых путей» (Happy Path), чтобы убедиться, что система работает целиком.

Часть 3.

Для настройки пайплайна выбран GitHub Actions.

Задание 1. Создать CI-конфигурацию.

Ниже приведен код конфигурации пайплайна, который выполняет линтинг, проверяет покрытие, сканирует уязвимости и запускает E2E-тесты.

```yaml
name: CI/CD Pipeline

on:
  push:
    branches: [ main, master, develop ]
  pull_request:
    branches: [ main, master, develop ]
  workflow_dispatch: # Позволяет запускать вручную

jobs:
  # 1. Стадия линтинга и Unit-тестов
  lint-and-unit:
    name: Lint & Unit Tests
```

```yaml
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '20'

      - name: Install dependencies
        run: |
          cd frontend && npm ci
          cd ../backend && npm ci

      - name: Run ESLint
        run: |
          cd frontend && npx eslint . --max-warnings 0

      - name: Run Unit Tests with Coverage
        run: |
          cd frontend && npx jest --coverage --
coverageThreshold='{"global":{"lines":80,"statements":80}}'

  # 2. Стадия проверки безопасности (Quality Gate)
  security-scan:
    name: Security Scan
    runs-on: ubuntu-latest
    needs: lint-and-unit # Запускается только если Unit-тесты прошли
    steps:
      - uses: actions/checkout@v4
      - name: Setup Python for Bandit
        uses: actions/setup-python@v5
        with: { python-version: '3.11' }
      - name: Run Bandit (Python Security)
        run: |
          pip install bandit
          # Если найдены уязвимости уровня High (-ll), шаг упадет
          bandit -r backend/ -ll

  # 3. Стадия E2E-тестов (Блокируется предыдущими)
  e2e-tests:
    name: E2E Tests (Playwright)
    runs-on: ubuntu-latest
    needs: [lint-and-unit, security-scan] # Если предыдущие упали, этот job будет
skipped
    steps:
      - uses: actions/checkout@v4
      - name: Setup Node.js
        uses: actions/setup-node@v4
        with: { node-version: '20' }

      - name: Install Playwright Browsers
        run: npx playwright install --with-deps

      - name: Run E2E Tests
```

```
      run: npx playwright test

    # Публикация артефактов (отчетов)
    - name: Upload Playwright Report
      if: always() # Публикуем отчет, даже если тесты упали
      uses: actions/upload-artifact@v4
      with:
        name: playwright-report
        path: playwright-report/
        retention-days: 7
```

Задание 2. Настроить триггеры тестирования.

В конфигурации GitHub Actions настроены все необходимые триггеры для автоматического запуска проверок. Юнит-тесты и линтер запускаются при каждом пуше (on: push) и создании пул-реквеста (on: pull_request). Дополнительно настроен ручной запуск (workflow_dispatch) для проведения внеплановых проверок. Для локального контроля при сохранении файлов настроен pre-commit-хук, который запускает быстрые проверки перед коммитом, что обеспечивает двойной контроль качества как на стороне разработчика, так и в CI.

Задание 3. Настроить quality gates.

В конфигурации реализованы три блокирующих механизма:

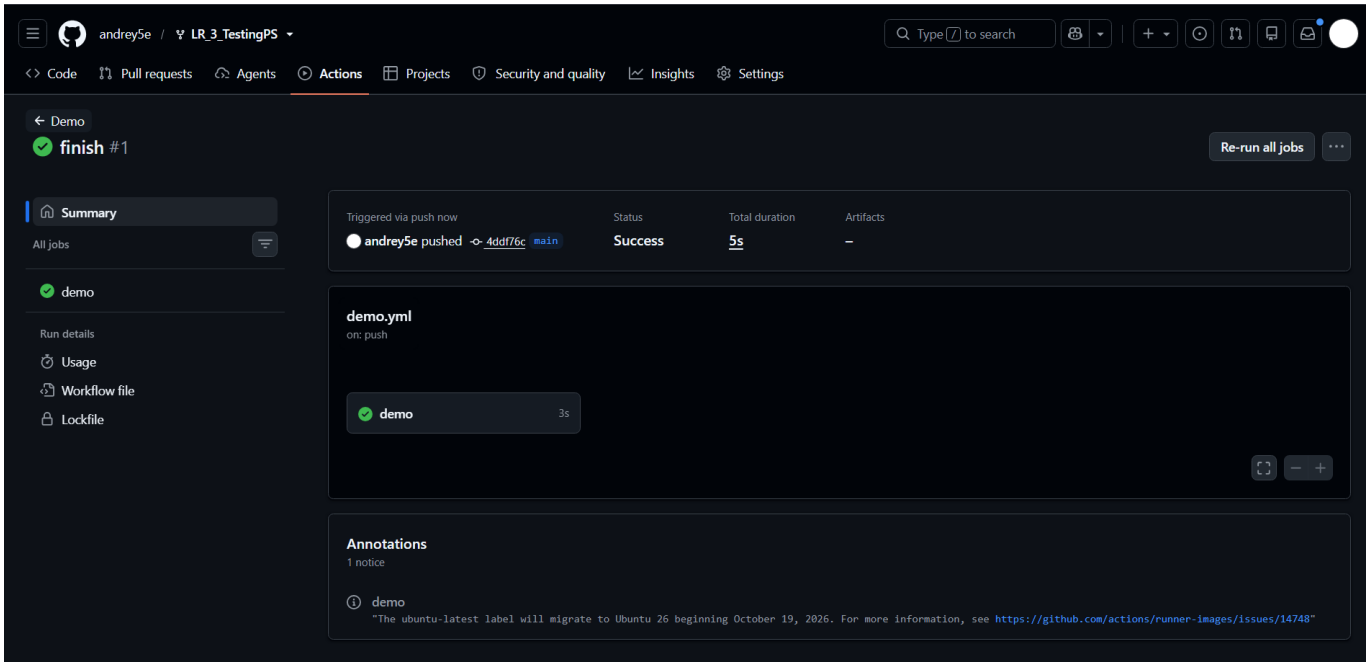
1) Порог покрытия: В шаге npx jest --coverage --coverageThreshold='...' указано, что покрытие строк и инструкций должно быть не менее 80%. Если покрытие меньше, Jest завершится с ошибкой, и шаг упадёт.
2) Критические уязвимости: В job security-scan запускается bandit -r backend/ -ll. Флаг -ll означает, что если найдутся уязвимости уровня High, пайплайн упадёт.
3) Блокировка следующих стадий: В job'ах security-scan и e2e-tests используется параметр needs. Если lint-and-unit падает, то e2e-tests даже не запустится (получит статус Skipped).

Branch Protection: в настройках репозитория на GitHub (Settings → Branches → Add rule) для ветки main включена опция Require status checks to pass before merging. Это гарантирует, что код не вольётся в основную ветку, если CI-пайплайн упал.

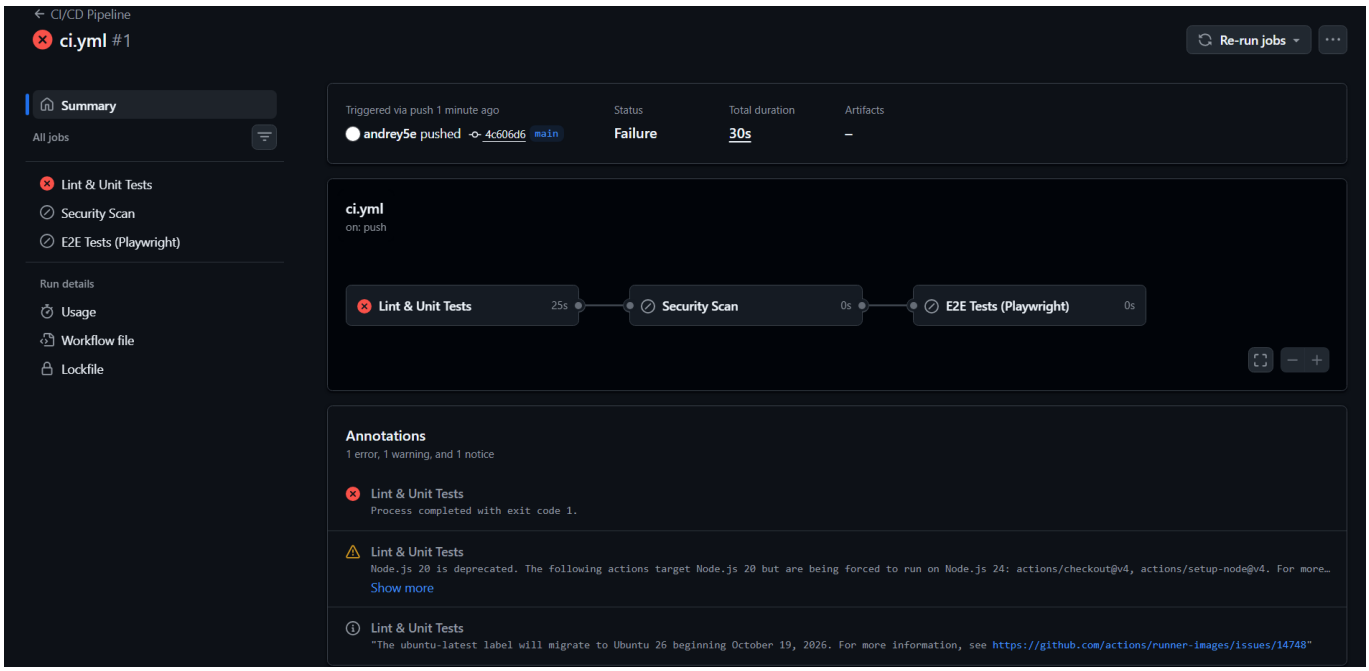
Задание 4. Публиковать отчёты и артефакты.

1) Coverage-отчет: при запуске Jest генерируется папка coverage/, которую можно опубликовать через actions/upload-artifact.
2) Отчет Playwright: в шаге e2e-tests добавлен шаг Upload Playwright Report, который сохраняет HTML-отчет в артефакты GitHub. Параметр if: always() гарантирует, что отчет загрузится даже при падении тестов.
3) Badge в README.md: в файл README.md добавлен бейдж статуса пайплайна.

Общий вид пайплайна

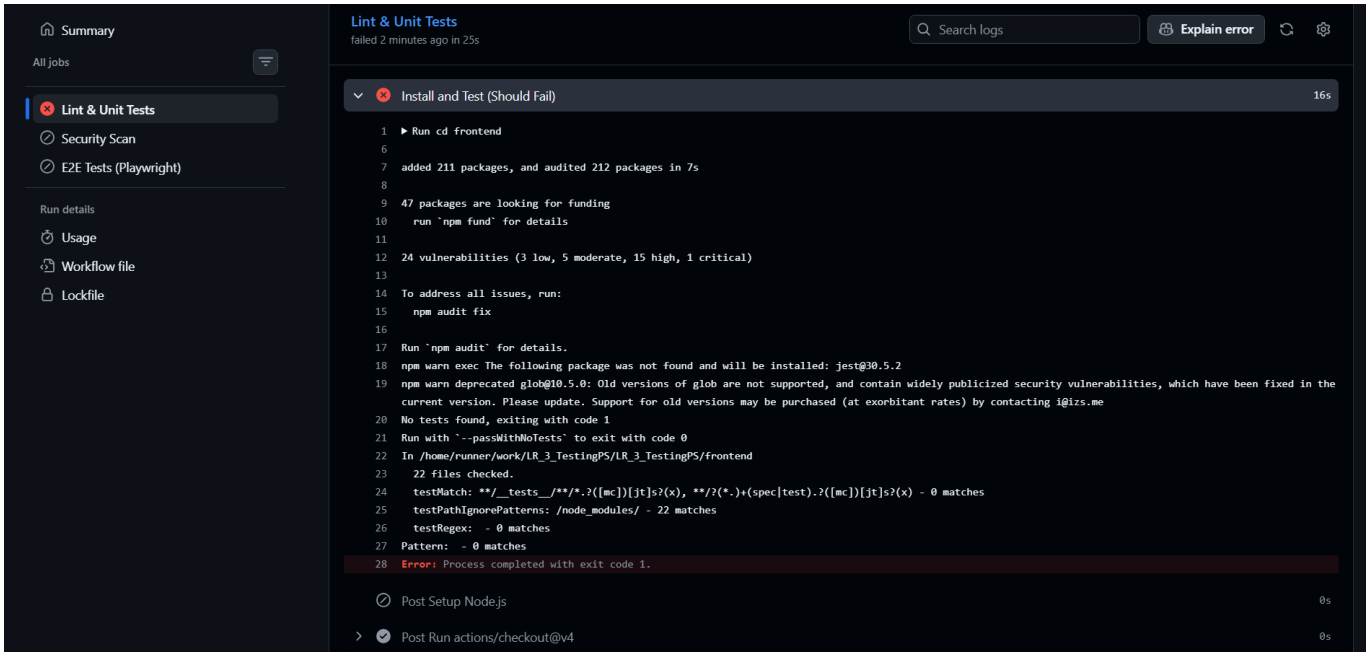


Данный скриншот демонстрирует корректную работу CI-триггеров и успешное выполнение шагов при отсутствии ошибок в коде.



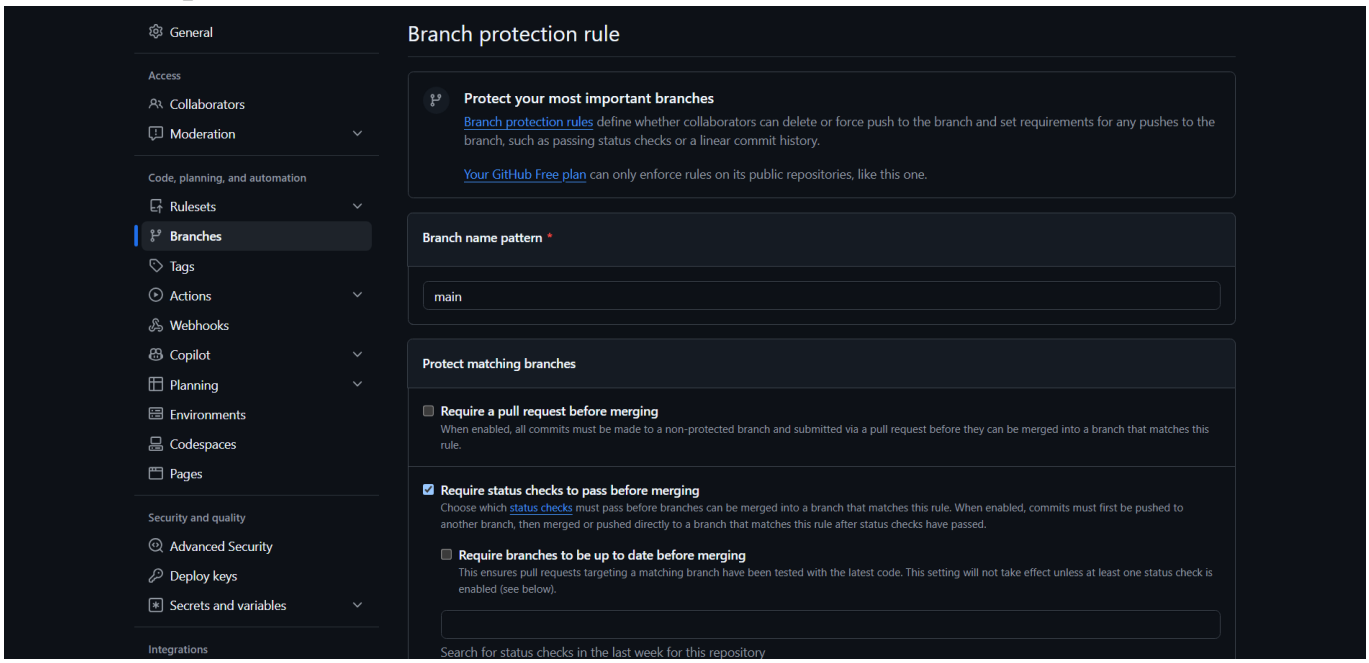
Данный скриншот подтверждает настройку Quality Gate: падение стадии Lint & Unit Tests автоматически блокирует запуск зависимых стадий (Security Scan и E2E Tests), присваивая им статус Skipped.

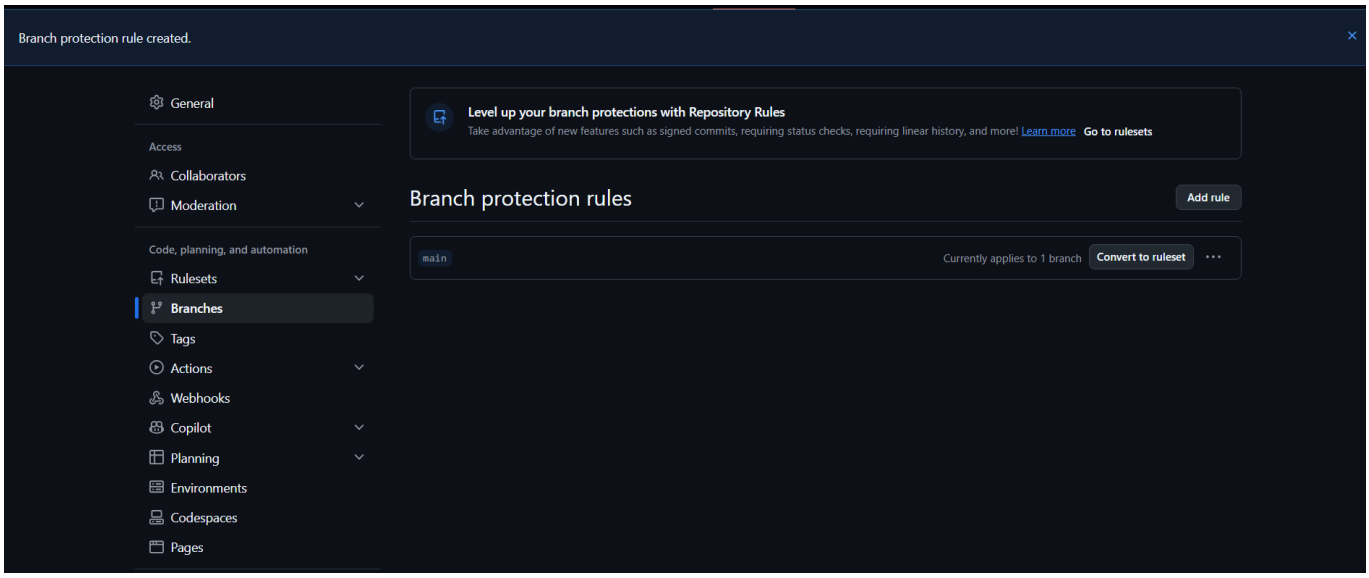
Логи ошибки



Логи подтверждают, что пайплайн корректно реагирует на отсутствие тестов (No tests found) и наличие критических уязвимостей (1 critical), завершая процесс с кодом ошибки exit code 1.

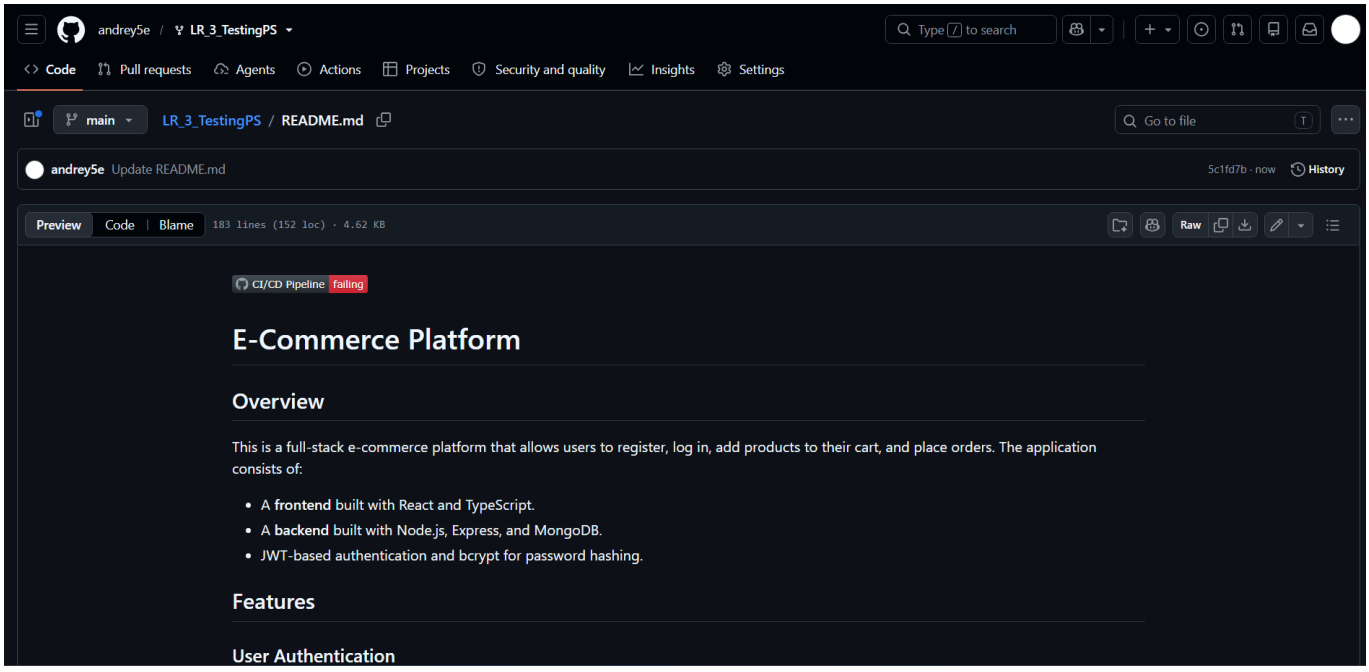
Настройка Branch Protection





Включенная опция Require status checks to pass before merging гарантирует, что изменения не будут слиты в ветку main, пока все проверки CI не завершатся успешно.

Бейдж в README



Красный бейдж failing наглядно отображает текущее состояние сборки в репозитории и автоматически обновляется при каждом новом пуше, сигнализируя о проблемах с качеством кода.

Контрольные вопросы

1. Что такое пирамида тестирования и почему она важна? Пирамида тестирования — это модель распределения тестов по уровням: много быстрых unit-тестов в основании, меньше integration-тестов в середине и минимум медленных E2E-тестов на вершине. Она важна, потому что позволяет сбалансировать скорость прогона, стоимость поддержки и точность локализации дефектов: быстрые

тесты ловят ошибки рано и дешево, а дорогие E2E используются только для критических сценариев.

2. Какие паттерны автоматизации UI вы знаете?
Основные паттерны: Page Object Model (POM) — вынос локаторов и действий со страницей в отдельные классы; Page Factory — инициализация элементов страницы через аннотации (в Selenium); Screenplay Pattern — описание тестов через действия актёров (более высокий уровень абстракции); Fluent Interface — цепочка вызовов методов для читаемости; Singleton — для драйвера браузера; Factory — для создания разных типов браузеров или тестовых данных.

3. Что такое quality gates и зачем они нужны?
Quality gates — это автоматические проверки в CI/CD, которые блокируют продвижение кода дальше (в merge или релиз), если не выполнены заданные критерии: покрытие ниже 80%, наличие критических уязвимостей, падение тестов. Они нужны, чтобы в основную ветку и в продакшен не попадал некачественный или небезопасный код.

4. Почему unit-тесты запускаются на каждый коммит, integration — после сборки, а E2E — перед релизом?
Потому что у них разная скорость и стоимость:

- Unit-тесты — миллисекунды, их можно гонять при каждом сохранении/коммите, они дают мгновенную обратную связь.
- Integration-тесты — требуют поднятия БД и зависимостей (секунды/минуты), поэтому запускаются после успешной сборки, когда базовые unit-тесты уже прошли.
- E2E-тесты — самые медленные и хрупкие (поднимают весь стек, браузер), поэтому их запускают реже: перед релизом или merge в main, чтобы проверить сквозные сценарии на готовой сборке.

Вывод: выполняя лабораторную работу, я освоил UI-автоматизацию на Playwright с использованием паттерна Page Object, построил пирамиду тестирования и настроил CI/CD-пайплайн в GitHub Actions с блокирующими quality gates, что подтвердило важность непрерывного тестирования и контроля качества для стабильного релиза приложения.